TP Unidad 7 - NoSQL y nuevas formas de almacenamiento: Food Store

Base de Datos II - Integrante: Mateo Algañaraz - Repositorio MateoAlganaraz/BaseDeDatos2, carpeta **tp9/** - PostgreSQL 16.15 y MongoDB 8.0.4 (mongosh 2.3.8)

1. Subconjunto y modelo documental (Consigna 1)

Tablas y claves en PostgreSQL: pedido (PK id_pedido; FK id_cliente → cliente), detalle_pedido (PK compuesta id_pedido + id_producto; FK a pedido y a producto; CHECK cantidad > 0 y precio > 0), cliente (PK id_cliente), producto (PK id_producto; FK id_categoria) y categoria. El esquema real no tiene una columna estado: el estado se deriva del borrado lógico (pedido.deleted_at nulo → VIGENTE, si no ANULADO), y una línea anulada es detalle_pedido.deleted_at no nulo.

Campo del documento (colección pedidos)	Origen en PostgreSQL	Tipo / decisión
_id	pedido.id_pedido	Long. **Identificador conservado**
fecha, forma_pago	pedido.fecha, pedido.forma_pago	Date, string
estado, anulado_en	pedido.deleted_at	VIGENTE / ANULADO; fecha de anulación
cliente {id_cliente, nombre, apellido, email}	cliente	**Objeto embebido** (snapshot); id_cliente conservado
lineas [{id_producto, producto, categoria, cantidad, precio_unitario, subtotal, anulada}]	detalle_pedido + producto + categoria	**Arreglo embebido**; id_producto conservado; **precio histórico** (precio_unitario cobrado) en Decimal128
total, cantidad_lineas_vigentes	Σ cantidad × precio_unitario de líneas no anuladas	Decimal128 calculado

2. Migración y reglas de integridad (Consignas 2 y 3)

Se migraron **12 pedidos** con 34 líneas, elegidos para cubrir todos los casos: 3 vigentes sin el producto 3373, 5 con el producto 3373, 3 anulados y el pedido 598881, cuya línea del producto 3373 está anulada. Son datos de laboratorio, sin personas reales. scripts/exportar_migracion.py genera migracion_mongodb.js (**insertMany**) desde la vista nosql_lab.documento_pedido. Resultado: "Documentos insertados: 12", con 12 documentos, 34 líneas e importe vigente total de 148.303,00.

Regla 1 - coherencia de importes: cada subtotal = cantidad × precio_unitario histórico, y total = suma de subtotales de líneas no anuladas. El precio que manda es el cobrado, no el actual: en PostgreSQL el producto 3373 hoy cuesta 1.351,08, pero en el pedido 2118 se cobró a 1.251,00. Por eso el total con precio actual (11.803,92) difiere del histórico (11.144,00). **Regla 2 - identidad y unicidad:** _id = id_pedido, cada id_producto aparece una sola vez por pedido (PK compuesta), cantidad > 0 y precio > 0. Ambas reglas se aplican en el motor con un validador ($jsonSchema + $expr). Tres documentos inválidos fueron rechazados con el código 121: total incoherente, producto repetido y cantidad 0.

Qué se embebió y por qué: las líneas y el cliente se embeben porque la aplicación lee el pedido completo y las líneas no existen fuera de él (relación "contiene", uno a pocos, unas 2,5 líneas por pedido). Así se lee y se escribe en una sola operación atómica, y el cliente y el producto quedan como foto del momento de la compra. **No se embebieron** el stock ni el precio de lista: son estado compartido entre pedidos y deben tener una única fuente.

```
== 3. Documento completo de ejemplo (pedido 2118) ==
{
  _id: Long('2118'),
  fecha: ISODate('2026-01-06T06:00:42.000Z'),
  estado: 'VIGENTE',
  anulado_en: null,
  forma_pago: 'EFECTIVO',
  cliente: {
    id_cliente: Long('5659'),
    nombre: 'Cliente05659',
    apellido: 'TP605659',
    email: 'tp6_cliente_05659@example.invalid'
  },
  lineas: [
    { id_producto: 3373, producto: 'TP6_PRODUCTO_03373', categoria: 'Almacén', cantidad: 3, precio_unitario: Decimal128('1251.00'), subtotal:
Decimal128('3753.00'), anulada: false },
    { id_producto: 5454, producto: 'TP6_PRODUCTO_05454', categoria: 'Almacén', cantidad: 2, precio_unitario: Decimal128('2248.00'), subtotal:
Decimal128('4496.00'), anulada: false },
    { id_producto: 7535, producto: 'TP6_PRODUCTO_07535', categoria: 'Mascotas', cantidad: 1, precio_unitario: Decimal128('2895.00'), subtotal:
Decimal128('2895.00'), anulada: false }
  ],
  total: Decimal128('11144.00'),
  cantidad_lineas_vigentes: 3
}
```

Evidencia de la colección: documento migrado del pedido 2118, tal como lo devuelve findOne() en MongoDB 8.0.4.

```
== 3. El validador rechaza documentos que violan las reglas (se esperan 3 rechazos) ==
RECHAZADO: total que no coincide con las líneas (regla 1) -> código 121: Document failed validation [$expr (reglas de total o productos repetidos)]
RECHAZADO: producto repetido en el mismo pedido (regla 2) -> código 121: Document failed validation [$expr (reglas de total o productos repetidos)]
RECHAZADO: cantidad 0 (regla 2) -> código 121: Document failed validation [$jsonSchema lineas.cantidad: minimum (comparison failed)]
```

Evidencia: rechazos del validador (evidencia/02_mongodb.txt).

3. Consultas y verificación (Consignas 4 y 5)

```
== 4.a Todos los pedidos con su identificador y estado ==
1   VIGENTE
2   VIGENTE
3   VIGENTE
7   ANULADO
57   ANULADO
107   ANULADO
2118   VIGENTE
5355   VIGENTE
5644   VIGENTE
8881   VIGENTE
12118   VIGENTE
598881   VIGENTE
```

MongoDB a): todos los pedidos con su identificador (_id = id_pedido) y estado.

```
== 4.b Pedidos con estado ANULADO ==
{
  cantidad: 3,
  ids: [7, 57, 107]
}
== 4.c (primera versión) Pedidos con el producto 3373 en el arreglo de líneas ==
{
  cantidad: 6,
  ids: [2118, 5355, 5644, 8881, 12118, 598881]
}
== 4.c (corregida) El producto 3373 en una línea NO anulada: $elemMatch sobre el mismo elemento ==
{
  cantidad: 5,
  ids: [2118, 5355, 5644, 8881, 12118]
}
```

MongoDB: b) estado ANULADO; c) primera versión y versión corregida (evidencia/02_mongodb.txt; la consulta a) lista los 12 _id con su estado).

```
== 4.d Pedidos con total > 20000 (umbral entre la mediana 14.487 y el P75 33.515 del subconjunto) ==
7   ANULADO   46524.00
5355   VIGENTE   44182.00
3   VIGENTE   42340.00
107   ANULADO   30574.00

== 4.d Solo pedidos VIGENTES con total > 20000 ==
5355   44182.00
3   42340.00
```

MongoDB d): total > 20.000. El umbral está entre la mediana (14.487) y el P75 (33.515) de los totales migrados: marca los pedidos grandes, con más de 1,4 veces la mediana. Devuelve 4 (2 vigentes).

```
== 4.b Pedidos con estado ANULADO, dentro del subconjunto migrado ==
 id_pedido
-----------
         7
        57
       107
(3 rows)

== 4.c Pedidos que contienen el producto 3373 en una línea vigente, dentro del subconjunto ==
 id_pedido
-----------
      2118
      5355
      5644
      8881
     12118
(5 rows)
```

PostgreSQL: consultas equivalentes a b) y c) sobre los pedidos migrados (evidencia/01_postgresql.txt).

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?
Estado seleccionado (ANULADO)	3 (7, 57, 107)	3 (7, 57, 107)	Sí
Producto seleccionado (3373), primera versión	5 (2118, 5355, 5644, 8881, 12118)	6 (los mismos + 598881)	**No**
Producto seleccionado (3373), corregida	5 (2118, 5355, 5644, 8881, 12118)	5 (2118, 5355, 5644, 8881, 12118)	Sí

Diferencias detectadas y corrección. (1) **Línea anulada:** la consulta en PostgreSQL filtra deleted_at IS NULL, pero la primera versión en MongoDB ({"lineas.id_producto": 3373}) también encontraba el pedido 598881, cuya línea está anulada. Se corrigió la consulta con $elemMatch: {id_producto: 3373, anulada: false}, que aplica ambas condiciones al mismo elemento del arreglo; usa el índice multiclave lineas.id_producto (IXSCAN, 6 examinados, 5 devueltos). El modelo no se cambió: conservar la línea anulada es correcto, y PostgreSQL también la conserva. (2) **Alcance:** sin restringir al subconjunto, PostgreSQL devuelve 12.000 pedidos anulados y 239 con el producto 3373. Por eso la comparación se hace sobre los 12 pedidos migrados (nosql_lab.pedido_migrado).

4. Evaluación con IA y contraste (Consignas 6 y 7)

Se consultó a Claude (Anthropic), en una instancia sin acceso al repositorio. El prompt describía el modelo actual, el acceso frecuente a pedidos completos y la coherencia de cantidades, precios y stock (ia/prompt_comparacion.md; respuesta completa en ia/respuesta_comparacion.md). Recomendó mantener PostgreSQL y usar MongoDB, a lo sumo, como modelo de lectura. Se contrastaron dos afirmaciones:

Afirmación de la IA	Documentación oficial	Laboratorio	Veredicto
"$jsonSchema no puede verificar que total sea la suma de los subtotales; un validador con $expr ayuda solo de forma limitada, y en la práctica se confía en el código."	Specify Validation With Query Operators: el ejemplo usa $expr para exigir totalWithVAT = total × (1 + VAT). Solo excluye $function, $where, $near y $text.	El validador con $expr rechazó un total incoherente al insertar y también un $inc de cantidad sin recalcular (código 121), y aceptó la misma actualización con subtotal y total recalculados.	**Imprecisa.** El motor sí garantiza la regla dentro del documento. Lo que no cubre es lo que cruza documentos (stock).
"La transacción multi-documento (que requiere un replica set) asegura que el pedido y los descuentos de stock se confirmen juntos."	Transactions: se soportan en replica sets y clusters fragmentados, con un costo mayor que las escrituras de un solo documento.	En standalone: error 20 "Transaction numbers are only allowed on a replica set member or mongos", y el stock queda intacto. En un replica set de un nodo: transacción confirmada, stock 5 → 3 y 1 pedido.	**Correcta.** Sumar MongoDB para pedidos y stock exige operar un replica set.

```
### 1. Servidor standalone (127.0.0.1:27017)
Servidor: MongoDB 8.0.4 (standalone)

== Afirmación 1: el validador también controla las ACTUALIZACIONES del total ==
{
  antes_total: '11144.00',
  antes_cantidades: [
    3,
    2,
    1
  ]
}
1.a RECHAZADO por el validador: código 121 - Document failed validation
1.b ACEPTADO: modificadas=1; total 11144.00 -> 12395.00 (+ 1251.00)
    revertido: total = 11144.00

== Afirmación 2: transacción multi-documento (pedido + descuento de stock) ==
Transacción FALLIDA: código 20 - Transaction numbers are only allowed on a replica set member or mongos
Estado tras el fallo: stock=5, pedidos=0

### 2. Replica set de un nodo (127.0.0.1:27018)
Servidor: MongoDB 8.0.4 (replica set "rs_tp9")

== Afirmación 2: transacción multi-documento (pedido + descuento de stock) ==
Transacción CONFIRMADA: stock=3, pedidos=1
```

Laboratorio de las dos afirmaciones (pruebas_afirmaciones_ia.js, evidencia/03_pruebas_afirmaciones_ia.txt).

Además, la IA propuso agregar CHECK (stock >= 0), una restricción que el esquema ya tiene (ck_producto_stock_no_negativo), y afirmó que a este volumen el rendimiento no decide. El laboratorio lo confirma: armar el pedido completo con JOIN y json_agg sobre 600.000 pedidos tarda 0,170 ms en PostgreSQL (tres Index Scan), y MongoDB lee el documento por _id (EXPRESS_IXSCAN, 1 examinado, 0 ms).

5. Conclusión

Conviene **mantener PostgreSQL como sistema de registro de los pedidos** y usar MongoDB, si hiciera falta, solo como modelo de lectura sincronizado. **Criterio 1, integridad transaccional:** las reglas críticas de Food Store cruzan entidades. Descontar stock y registrar el pedido debe ser atómico, y PostgreSQL lo garantiza con ACID, FK y CHECK. En MongoDB, el laboratorio mostró que esa transacción falla en un servidor standalone y exige un replica set. **Criterio 2, patrón de acceso y rendimiento:** el documento embebido refleja bien la lectura del pedido completo y el validador con $expr protege total y líneas dentro del documento. Pero a este volumen PostgreSQL arma el mismo pedido en 0,17 ms con json_agg, así que no hay una ganancia que justifique duplicar datos. **Criterio 3, operación:** un segundo motor suma sincronización, respaldos y monitoreo. Combinar tecnologías se justificaría solo si la lectura del historial se vuelve un cuello de botella medido o si el esquema de los pedidos necesita variar mucho.

Archivos: consultas_mongodb.js, consultas_postgresql.sql, migracion_mongodb.js, pruebas_afirmaciones_ia.js, lectura_pedido_completo.sql/.js, scripts/ y evidencia/ (salidas completas). La herramienta de IA (Claude) también asistió en la redacción de los scripts; cada uno se verificó ejecutándolo en ambos motores.